

Movie Database Management System

A fully functional web-based Movie Database Management System with a responsive admin dashboard was developed. The system enables administrators and users to manage movies, actors, directors, reviews, ratings, and watchlists efficiently. It supports advanced search, filtering, analytics visualization, and recommendation capabilities using MongoDB aggregation pipelines. The platform provides real-time insights into movie popularity, top-rated content, genre distribution, and user engagement through an interactive dashboard.

Output

```
[revlin@revlin movie-db-management-system]$ curl -X POST "http://127.0.0.1:8000/api/movies" -H "Content-Type: application/json" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOiE3ODc0MTE5NDUsInN1YiI6IjZmZGZyZ2IwYmFkODMwNWY2ZTU1ZjU5ZCIsInR5cGU0IjJhY2Nlc3MiIiwgXzYBcF05L0y5B40HI_bot58RkP_Jt32RXazB6gQb8" -d '{
  "title": "CRUD Demo Movie",
  "description": "Testing create movie",
  "release_year": 2026,
  "duration_minutes": 120,
  "language": "English",
  "genres": ["Action", "Demo"]
}'
```

test movie creation. Just does a post command where we create a movie entry and store it in mongodb.

```
[revlin@revlin movie-db-management-system]$ curl "http://127.0.0.1:8000/api/movies/?search=CRUD&page=1&per_page=5"
[{"id": "6a343be92da9abefd793fb2c", "title": "CRUD Smoke Test Movie", "description": "Created from curl smoke test", "release_year": 2026, "duration_minutes": 111, "language": "English", "genres": ["Drama", "Test"], "director_id": null, "actor_ids": [], "poster_url": null, "created_at": "2026-06-18T18:49:51.111000"}][revlin@revlin movie-db-management-system]$
```

Fetching the entry from the db

```
[revlin@revlin movie-db-management-system]$ curl -X PUT "http://127.0.0.1:8000/api/movies/6a343be92da9abefd793fb2c" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOiE3ODc0MTE5NDUsInN1YiI6IjZmZGZyZ2IwYmFkODMwNWY2ZTU1ZjU5ZCIsInR5cGU0IjJhY2Nlc3MiIiwgXzYBcF05L0y5B40HI_bot58RkP_Jt32RXazB6gQb8" \
-d '{
  "title": "CRUD Demo Movie Updated",
  "release_year": 2027
}'
{"id": "6a343be92da9abefd793fb2c", "title": "CRUD Demo Movie Updated", "description": "Created from curl smoke test", "release_year": 2027, "duration_minutes": 111, "language": "English", "genres": ["Drama", "Test"], "director_id": null, "actor_ids": [], "poster_url": null, "created_at": "2026-06-18T18:49:51.111000"}[revlin@revlin movie-db-management-system]$
```

Updating the movie details

```
[revlin@revlin movie-db-management-system]$ curl -X DELETE "http://127.0.0.1:8000/api/movies/6a343be92da9abefd793fb2c" \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOiE3ODc0MTE5NDUsInN1YiI6IjZmZGZyZ2IwYmFkODMwNWY2ZTU1ZjU5ZCIsInR5cGU0IjJhY2Nlc3MiIiwgXzYBcF05L0y5B40HI_bot58RkP_Jt32RXazB6gQb8"
{"status": "deleted"}[revlin@revlin movie-db-management-system]$
```

deleting the record from the db

swagger docs

movies ^		
POST	/api/movies/ Create	  ▼
GET	/api/movies/ List All	▼
GET	/api/movies/{movie_id} Get One	▼
PUT	/api/movies/{movie_id} Update	 ▼
DELETE	/api/movies/{movie_id} Remove	 ▼

Movie DB Management System 0.1.0 OAS 3.1

/openapi.json

Authorize 

health ^

GET /api/health/ Health check ▼

auth ^

POST /api/auth/register Register ▼

POST /api/auth/login Login ▼

POST /api/auth/refresh Refresh ▼

Tools Used with Version

- Python 3.12
- FastAPI 0.115+
- MongoDB 8.0+
- Beanie ODM 1.29+
- Pydantic 2.10+
- Jinja2 3.1+
- Bootstrap 5.3+
- Chart.js 4.4+
- DataTables 2.0+
- Uvicorn 0.34+
- Docker 27.0+
- JWT Authentication Library (python-jose 3.3+)
- Bcrypt 4.2+

Summary

The Movie Database Management System was designed to centralize the storage, management, and retrieval of movie-related information. The application includes modules for user authentication, movie management, actor and director management, reviews and ratings, watchlist management, and analytics. MongoDB was used as the primary database due to its flexible document-based schema, making it suitable for storing complex movie-related data. FastAPI was used to build scalable backend APIs, while Jinja2 templates with Bootstrap and Chart.js were used to create a responsive dashboard interface. Additional features such as pagination, search optimization, filtering, and aggregation pipelines improved performance and usability.

Conclusion

The project successfully demonstrates the implementation of a scalable and efficient movie management platform using modern web technologies. It highlights practical usage of MongoDB features such as indexing, aggregation pipelines, and document relationships while showcasing backend development, dashboard creation, and analytics visualization. The system serves as a robust solution for managing large volumes of movie data and provides a strong foundation for future enhancements such as AI-based recommendations and advanced personalization.